

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



INTERNSHIP 1 REPORT

**DEVELOPMENT OF A REPRODUCIBLE
CREDIT CARD FRAUD DETECTION PIPELINE**

Full name : Dang The Dong A
Student ID : 2570791
Major : Computer Science
Course : Internship 1
Instructor : Prof. Quan Thanh Tho
Submission date : 30 April 2026

Ho Chi Minh City, 2026

Acknowledgement

I would like to express my sincere appreciation to the lecturers of Ho Chi Minh City University of Technology, and especially to my instructor, Prof. Quan Thanh Tho, for their guidance during the completion of this Internship 1 project. Their comments and academic direction were valuable in refining both the technical scope and the presentation of the work.

I also acknowledge the open-source community for the software libraries used in this project, particularly Python, scikit-learn, imbalanced-learn, Matplotlib, and Streamlit. These tools provided a reliable foundation for implementing and evaluating the proposed workflow.

Abstract

This report presents the design and implementation of a reproducible machine learning pipeline for credit card fraud detection using the public European cardholder transaction dataset. The objective of the project was not limited to training a classifier; rather, it was to develop a compact engineering workflow that supports data preparation, model training, evaluation, experiment tracking, and result inspection in a systematic manner.

The implemented system includes temporal train-test splitting, feature scaling, class imbalance handling with SMOTE, training of four model families (logistic regression, random forest, XGBoost, and LightGBM), hyperparameter sweep, run comparison, and a lightweight dashboard for reviewing stored experiment artifacts. Logistic regression and random forest provide interpretable baselines, while gradient boosting methods are included because they are widely used in tabular fraud detection practice [10, 11, 12].

Evaluation was carried out using precision, recall, F1-score, and average precision on the same temporal holdout set at threshold $\tau = 0.5$. Among the evaluated runs, the strongest random forest configuration achieved the strongest overall trade-off with precision 0.9825, recall 0.7467, F1-score 0.8485, and average precision 0.8218. Logistic regression achieved higher recall (0.8800) but substantially lower precision (0.0676) at the fixed threshold. These results show that extending the pipeline beyond classical baselines improves empirical comparison while preserving reproducibility.

Contents

Acknowledgement	i
Abstract	ii
1 Introduction	1
1.1 Background	1
1.2 Objectives	1
1.3 Scope	2
1.4 Report Organization	2
2 Assigned Tasks and Work Plan	3
2.1 Assigned Tasks	3
2.2 Work Plan	3
2.3 Expected Deliverables	4
3 Theoretical Background	5
3.1 Credit Card Fraud Detection	5
3.2 Imbalanced Classification	5
3.3 Evaluation Metrics	5
3.4 Temporal Holdout Strategy	7
3.5 Models	7
3.5.1 Logistic Regression	7
3.5.2 Random Forest	7
3.5.3 XGBoost	7
3.5.4 LightGBM	8
4 Requirement Analysis and System Design	9

4.1	Functional Requirements	9
4.2	Non-Functional Requirements	9
4.3	Overall Workflow	10
4.4	Module Organization	10
4.5	Artifact Design	11
5	Implementation	12
5.1	Development Environment	12
5.2	Dataset Preparation	12
5.3	Training Workflow	13
5.4	Evaluation Workflow	13
5.5	Hyperparameter Sweep and Comparison Support	13
5.6	Dashboard Interface	13
6	Experimental Setup and Results	14
6.1	Dataset Summary	14
6.2	Model Configurations	14
6.3	Quantitative Results	15
6.4	Confusion Matrices	15
6.5	Precision-Recall Curves	18
6.6	Analysis of Results	20
7	Discussion	21
7.1	Main Contributions of the Project	21
7.2	Limitations	21
7.3	Learning Outcomes	22
7.4	Personal Contribution Summary	22
8	Conclusion and Future Work	23
8.1	Conclusion	23
8.2	Future Work	24
A	Example Commands	27

List of Figures

6.1	Confusion matrix for the logistic regression run 20260430T113650Z-b4177ef2 at threshold 0.5	16
6.2	Confusion matrix for the random forest run 20260430T085742Z-e128abd4 at threshold 0.5	16
6.3	Confusion matrix for the XGBoost run 20260518T034401Z-bfbd12a2 at threshold 0.5	17
6.4	Confusion matrix for the LightGBM run 20260518T034536Z-d5840edb at threshold 0.5	17
6.5	Precision-recall curve for the logistic regression run 20260429T071627Z-bdf01b47	18
6.6	Precision-recall curve for the random forest run 20260429T162247Z-c0a205a5	18
6.7	Precision-recall curve for the XGBoost run 20260518T034401Z-bfbd12a2	19
6.8	Precision-recall curve for the LightGBM run 20260518T034536Z-d5840edb	19

List of Tables

6.1	Prepared dataset summary	14
6.2	Model performance on the temporal holdout set	15

Chapter 1

Introduction

This chapter introduces the fraud detection problem context, states the objectives and scope of the project, and explains how the report is organized.

1.1 Background

Credit card fraud detection is a representative problem in applied machine learning, especially in domains where the target class is rare and the cost of misclassification is high [10]. In this setting, a useful solution requires more than a single trained model. It must also incorporate correct preprocessing, leakage prevention, reproducibility, and clear evaluation procedures.

The present project was developed as a compact fraud detection workflow based on the public `creditcard.csv` dataset. The emphasis of the project is placed on sound engineering practice: modular implementation, deterministic execution, explicit artifact management, and evaluation methods that are appropriate for highly imbalanced data.

1.2 Objectives

The objectives of the project are as follows:

1. to develop a modular pipeline for credit card fraud detection;
2. to apply a temporal train-test split in order to reduce information leakage;
3. to perform preprocessing and imbalance handling in a reproducible manner;
4. to compare baseline models using suitable metrics for imbalanced classification;
5. to preserve experiment outputs for subsequent inspection and comparison.

1.3 Scope

The scope of the work includes data preparation, model training, evaluation, hyperparameter sweep, run comparison, and dashboard-based inspection of saved results. The project does not include online inference, large-scale deployment, enterprise integration, or real-time fraud response. Its primary purpose is to demonstrate a complete and well-structured machine learning workflow at the scale appropriate to Internship 1.

1.4 Report Organization

The remainder of this report is organized as follows. Chapter 2 presents the assigned tasks and work plan. Chapter 3 reviews the theoretical background relevant to the project. Chapter 4 discusses the requirements and overall system design. Chapter 5 describes the implementation in detail. Chapter 6 presents the experimental setup and evaluation results. Chapter 7 discusses the significance and limitations of the work. Chapter 8 concludes the report and outlines possible future extensions.

Chapter 2

Assigned Tasks and Work Plan

This chapter summarizes the assigned internship tasks, the planned implementation stages, and the main deliverables expected from the project.

2.1 Assigned Tasks

The project was carried out through the following main tasks:

1. studying the characteristics of the fraud detection problem and the public dataset used in the project;
2. implementing a preprocessing stage that validates the data, sorts transactions by time, and creates a temporal split;
3. applying feature scaling and oversampling to the training data in a controlled manner;
4. training and evaluating baseline classification models;
5. generating experiment artifacts such as reports, curve data, and image outputs;
6. tracking multiple experimental runs for later comparison;
7. developing a simple dashboard for reviewing saved runs.

2.2 Work Plan

The work was organized into the following stages:

1. problem study and dataset familiarization;
2. preprocessing pipeline design and implementation;

3. model training and evaluation implementation;
4. experiment tracking and comparison support;
5. dashboard development;
6. documentation and report preparation.

2.3 Expected Deliverables

The final output of the project consists of:

- a modular Python implementation under `src/`;
- processed dataset artifacts under `data/`;
- model, report, sweep, and run-history artifacts under `artifacts/`;
- a Streamlit dashboard under `ui/app.py`;
- project documentation in `README.md`.

Chapter 3

Theoretical Background

This chapter reviews the theoretical concepts that support the implementation, including imbalanced classification, evaluation metrics, temporal validation, and model families used in the study.

3.1 Credit Card Fraud Detection

Credit card fraud detection aims to distinguish fraudulent transactions from legitimate ones in a highly skewed dataset. In practice, the number of fraudulent transactions is very small compared with the number of legitimate transactions. This imbalance makes the problem challenging because a model may achieve a high overall accuracy while still failing to detect many fraud cases.

3.2 Imbalanced Classification

In imbalanced classification, the minority class is underrepresented relative to the majority class. Standard learning algorithms often become biased toward the majority class, especially when the class distribution is extreme. In the context of fraud detection, such bias is undesirable because missed fraud cases may be costly.

To mitigate this issue, the project applies SMOTE to the training partition [7]. SMOTE creates synthetic samples for the minority class in feature space, thereby improving the representation of fraudulent transactions during model fitting.

3.3 Evaluation Metrics

Accuracy alone is not sufficient for this problem. Therefore, the evaluation stage computes metrics from confusion-matrix counts and score-based ranking.

Let the model output a fraud score $\hat{p}_i$ for each transaction i , and let the decision threshold be τ (in this report, $\tau = 0.5$ for threshold-based metrics). The predicted label is:

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq \tau \\ 0, & \hat{p}_i < \tau \end{cases}$$

where 1 denotes fraud and 0 denotes non-fraud.

From $\hat{y}_i$ and true label y_i , the following counts are obtained:

- *TP*: fraud correctly predicted as fraud;
- *FP*: non-fraud incorrectly predicted as fraud;
- *FN*: fraud incorrectly predicted as non-fraud.

The threshold-based metrics are then executed by the following formulas:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Average precision is computed from the full precision-recall curve by sorting transactions by score $\hat{p}_i$, sweeping all unique thresholds, and integrating precision as recall increases:

$$\text{AP} = \sum_{k=1}^n (R_k - R_{k-1}) P_k$$

where P_k and R_k are precision and recall at operating point k on the precision-recall curve.

In implementation terms, metric execution follows four steps: (1) generate scores $\hat{p}_i$, (2) apply threshold τ to obtain $\hat{y}_i$, (3) compute *TP*, *FP*, and *FN*, and (4) calculate Precision, Recall, F1-score, and AP using the formulas above. These metrics provide a more meaningful assessment in imbalanced settings, where false negatives and false positives do not have the same practical impact. This focus is consistent with established analysis of the precision-recall view for highly skewed class distributions [8].

3.4 Temporal Holdout Strategy

The project uses the `Time` column to construct a temporal split. Earlier observations are used for training, while later observations are reserved for testing. This choice is more realistic than a random split because it approximates the way a deployed model would be evaluated on future transactions and helps reduce leakage risk that can appear when temporal structure is ignored [9].

3.5 Models

Four model families are evaluated on the same prepared data and temporal holdout protocol. In particular, the boosted tree families follow widely used formulations from XGBoost and LightGBM [11, 12].

3.5.1 Logistic Regression

For binary fraud detection, logistic regression models the posterior probability of fraud through a linear decision function followed by the logistic link:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

This formulation is computationally efficient, easy to regularize, and its coefficients are relatively interpretable, making it a suitable first baseline for tabular classification tasks [5, 13].

3.5.2 Random Forest

Random forest combines predictions from many decorrelated decision trees that are trained on bootstrap samples and randomized feature subsets at each split [6]. Averaging across trees usually reduces variance and improves robustness compared with a single tree, while still capturing nonlinear patterns and feature interactions that a linear model may miss. This complementary behavior is the main reason random forest is selected as the second baseline in this report.

3.5.3 XGBoost

XGBoost is a gradient boosted decision tree ensemble that fits trees sequentially to correct the residuals of previous trees [11]. It is well suited to tabular data and is com-

monly reported in fraud detection benchmarks. In this project, `XGBClassifier` is used with a fixed random seed, default learning rate 0.05, and regularization via subsampling and column subsampling.

3.5.4 LightGBM

LightGBM grows trees leaf-wise using a histogram-based algorithm, which often yields fast training on large tabular datasets [12]. The project uses `LGBMClassifier` with `class_weight = balanced` to mitigate class imbalance during boosting. Like XGBoost, it captures nonlinear patterns and feature interactions beyond a linear baseline.

Chapter 4

Requirement Analysis and System Design

This chapter defines the functional and non-functional requirements and presents the system workflow, module responsibilities, and artifact organization.

4.1 Functional Requirements

The system is required to perform the following functions:

- load a local dataset or download the default dataset automatically;
- validate the expected columns and sort transactions by time;
- split the dataset into training and test partitions according to temporal order;
- scale input features using statistics learned only from the training partition;
- resample the training data using SMOTE;
- train one or more classification models;
- evaluate saved models on an untouched temporal holdout set;
- conduct hyperparameter sweeps on an inner validation split;
- compare stored runs and display their results through a dashboard.

4.2 Non-Functional Requirements

The most important non-functional requirements are reproducibility, modularity, maintainability, and usability. These are addressed through a separated `src/` structure, a fixed

random seed, explicit artifact persistence, run manifests, and a command-line interface that supports repeated execution.

4.3 Overall Workflow

The system is organized into six user-facing stages:

1. `prepare`;
2. `train`;
3. `evaluate`;
4. `sweep`;
5. `compare`;
6. `dashboard`.

The first four stages create experimental outputs, while the last two stages support inspection and comparison of previously saved runs.

4.4 Module Organization

The implementation is divided into specialized modules:

- `src/data_processing.py` manages dataset loading, validation, temporal splitting, scaling, resampling, and metadata persistence;
- `src/model.py` defines the supported model families and sweep search spaces;
- `src/pipeline.py` coordinates the main workflows for training, evaluation, sweeping, and comparison;
- `src/evaluation.py` computes performance metrics and saves precision-recall artifacts;
- `src/cli.py` exposes the workflow through command-line commands;
- `ui/app.py` implements a dashboard for reviewing stored runs.

4.5 Artifact Design

Outputs are organized into separate directories for processed data, trained models, reports, sweeps, and run manifests. This structure prevents accidental overwriting of previous experiments and supports systematic comparison across runs.

Chapter 5

Implementation

This chapter describes the practical implementation details of the pipeline, from development environment and data preparation to training, evaluation, sweeping, and dashboard integration.

5.1 Development Environment

The project is implemented in Python and relies on Pandas, scikit-learn, imbalanced-learn, XGBoost, LightGBM, Matplotlib, Joblib, and Streamlit. Dependency versions are specified in `requirements.txt`. XGBoost and LightGBM require a system OpenMP runtime on some platforms; the project README documents local setup and a Docker-based alternative for reproducible execution.

5.2 Dataset Preparation

The input dataset contains anonymized numerical features together with the `Time`, `Amount`, and `Class` columns. During preparation, the system validates the required schema, sorts transactions by `Time`, creates the temporal split, applies feature scaling, and stores the processed data and metadata on disk.

The prepared dataset used in this report contains 227,845 training rows and 56,962 test rows. After resampling, the training partition contains 284,285 rows. The target variable is `Class`, and the feature set contains 30 columns, including `Time`, `Amount`, and the anonymized variables `V1` to `V28`.

5.3 Training Workflow

The training stage loads the prepared training data, constructs the requested model with the specified hyperparameters, fits the model, and stores the resulting artifact under a unique run directory. A training summary file is also generated to record the resolved parameters, training size, feature count, and random seed. This mechanism is important for repeatability and later analysis.

5.4 Evaluation Workflow

The evaluation stage loads a stored model, generates fraud scores for the temporal holdout set, applies a decision threshold, and computes precision, recall, F1-score, and average precision. In addition, it saves a JSON evaluation report, a precision-recall curve in CSV form, a PNG figure, an updated run manifest, and an updated run-history index.

5.5 Hyperparameter Sweep and Comparison Support

The sweep stage creates an inner temporal validation split from the training timeline and evaluates candidate hyperparameter combinations according to average precision, F1-score, and recall. The comparison stage then aggregates all completed runs into a sortable summary table. This design makes it possible to inspect the evolution of experiments without reopening each run manually.

5.6 Dashboard Interface

The project includes a lightweight Streamlit dashboard that reads the saved run history and manifests. The dashboard displays overall run statistics, detailed information for individual runs, and side-by-side comparisons. Although it is not essential for the core pipeline, it improves the usability of the project and supports demonstration in an academic setting.

Chapter 6

Experimental Setup and Results

This chapter presents the experimental configuration, summarizes the datasets and model settings, and reports quantitative and visual evaluation results on the temporal holdout set.

6.1 Dataset Summary

Table 6.1 summarizes the main dataset settings used in the prepared data.

Table 6.1: Prepared dataset summary

Item	Value
Training rows	227,845
Test rows	56,962
Resampled training rows	284,285
Feature count	30
Target column	Class
Temporal split start of test set	145248.0
Sampling strategy	0.25
Random seed	42
Fraud cases in evaluated test set	75

6.2 Model Configurations

The experiments include the following representative configurations (defaults unless sweep-tuned):

- **Logistic regression:** `C = 0.5, max_iter = 3000, solver = liblinear, class_weight = balanced;`
- **Random forest:** `n_estimators = 500, max_depth = 20, class_weight = balanced_subsamp`

- **XGBoost:** `n_estimators = 300, max_depth = 6, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 0.8;`
- **LightGBM:** `n_estimators = 300, num_leaves = 31, learning_rate = 0.05, class_weight = balanced, subsample = 0.8.`

All four models were evaluated at threshold $\tau = 0.5$ on the same temporal holdout partition. Although the run history also contains experiments at alternative thresholds, the summary table in this chapter intentionally reports representative $\tau = 0.5$ runs so that model behavior is compared under a single operating point.

6.3 Quantitative Results

Table 6.2 presents the principal evaluation metrics extracted from the saved evaluation artifacts and run history.

Table 6.2: Model performance on the temporal holdout set

Model	Precision	Recall	F1-score	Average Precision
Logistic Regression	0.0676	0.8800	0.1255	0.7706
Random Forest	0.9825	0.7467	0.8485	0.8218
XGBoost	0.8507	0.7600	0.8028	0.7830
LightGBM	0.7534	0.7333	0.7432	0.7811

6.4 Confusion Matrices

To complement the aggregate metrics in Table 6.2, Figures 6.1–6.4 present confusion matrices at the same decision threshold $\tau = 0.5$.

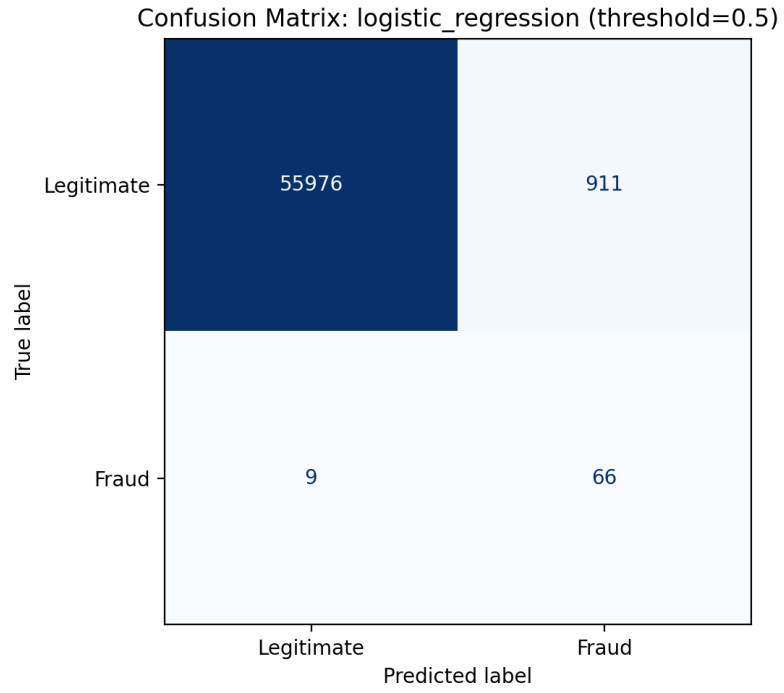


Figure 6.1: Confusion matrix for the logistic regression run 20260430T113650Z-b4177ef2 at threshold 0.5

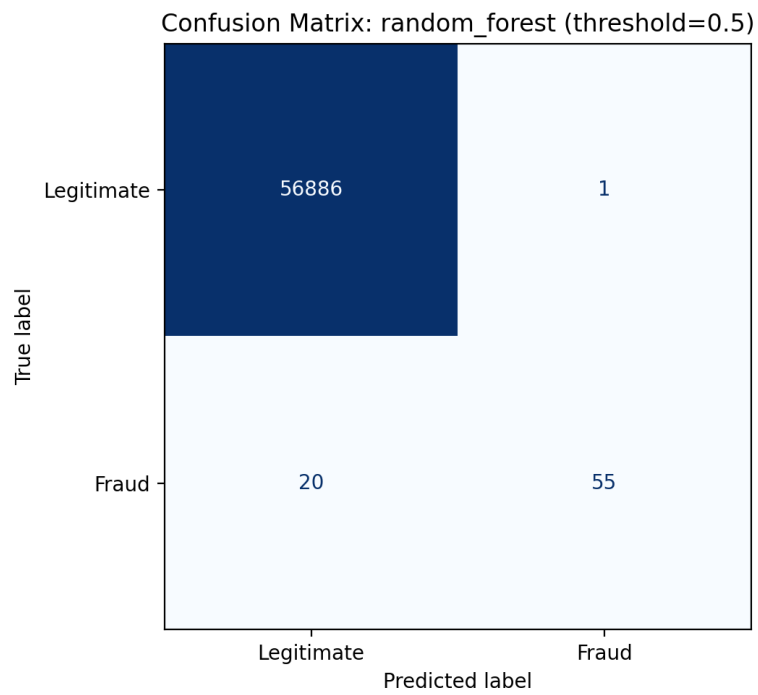


Figure 6.2: Confusion matrix for the random forest run 20260430T085742Z-e128abd4 at threshold 0.5

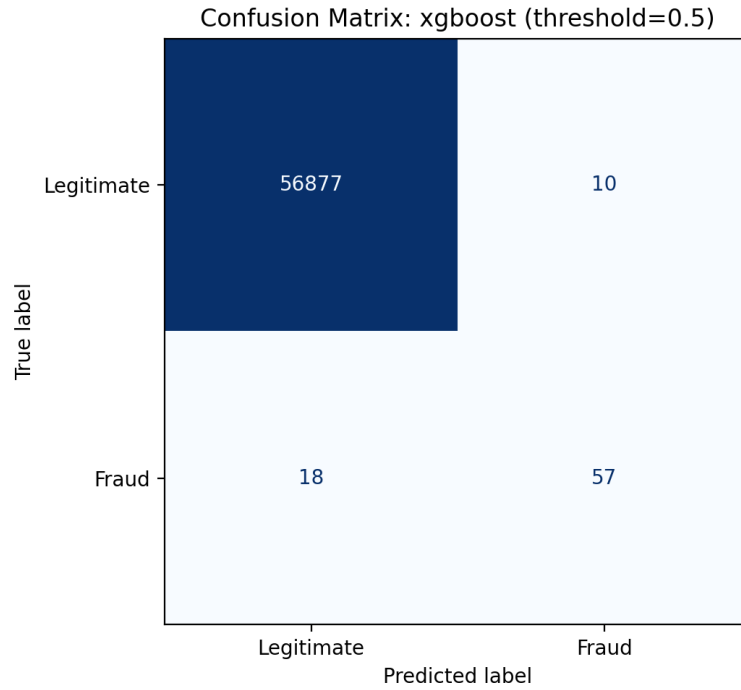


Figure 6.3: Confusion matrix for the XGBoost run 20260518T034401Z-bfbd12a2 at threshold 0.5

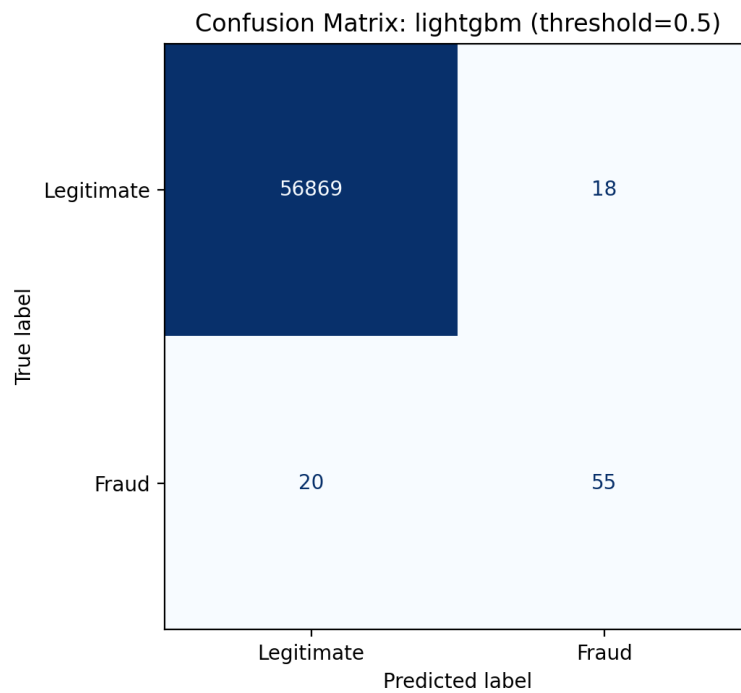


Figure 6.4: Confusion matrix for the LightGBM run 20260518T034536Z-d5840edb at threshold 0.5

6.5 Precision-Recall Curves

Figures 6.5–6.8 illustrate the saved precision-recall curves for representative runs of all four model families.

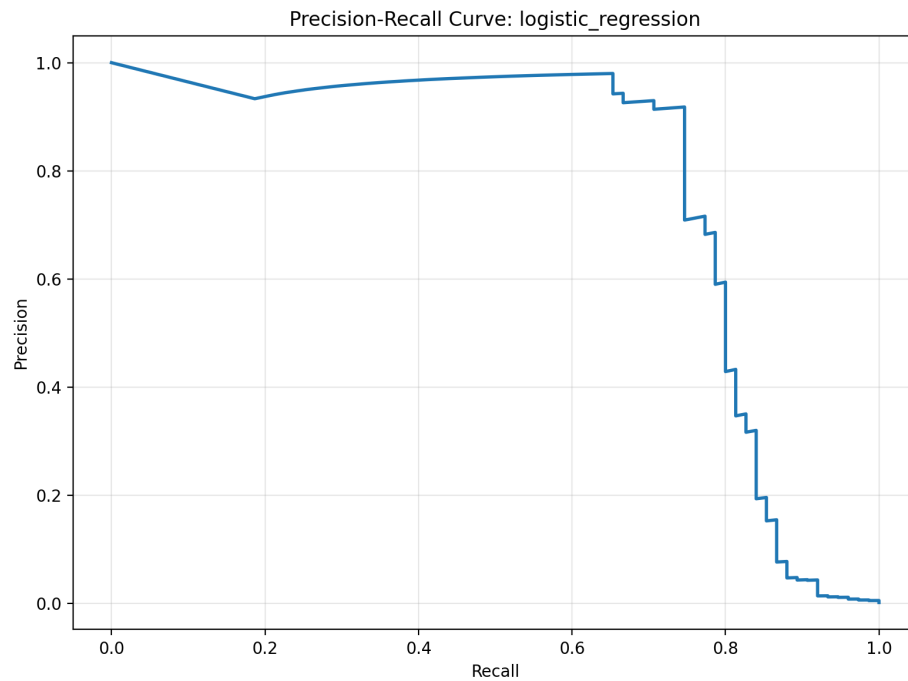


Figure 6.5: Precision-recall curve for the logistic regression run 20260429T071627Z-bdf01b47

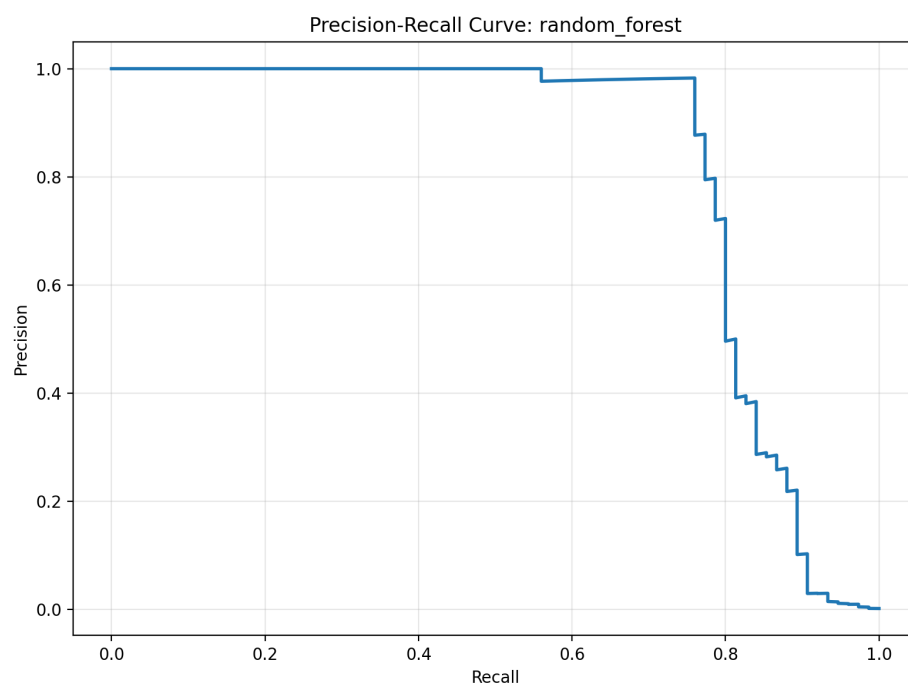


Figure 6.6: Precision-recall curve for the random forest run 20260429T162247Z-c0a205a5

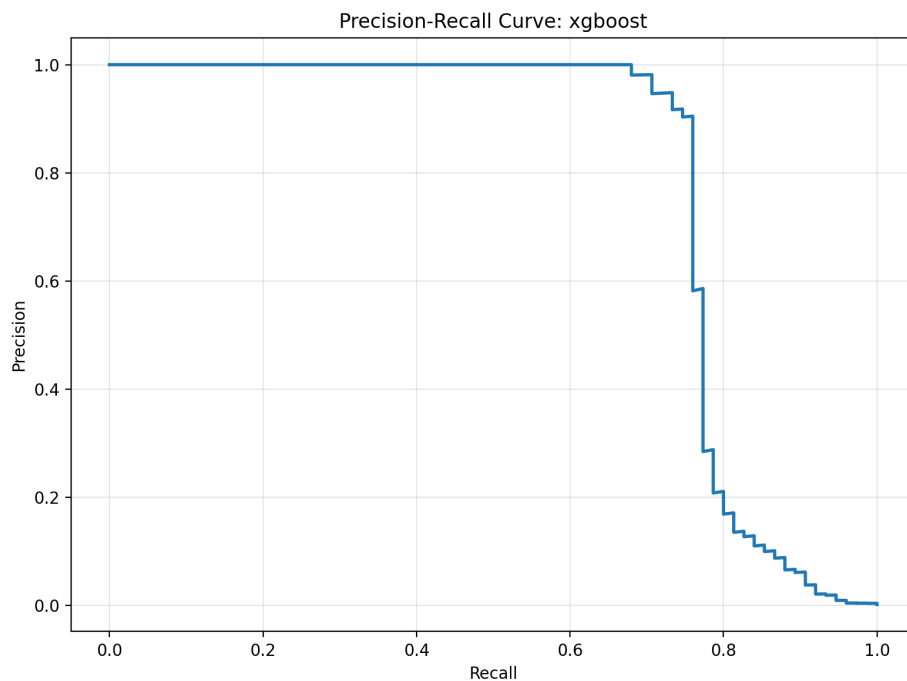


Figure 6.7: Precision-recall curve for the XGBoost run 20260518T034401Z-bfbd12a2

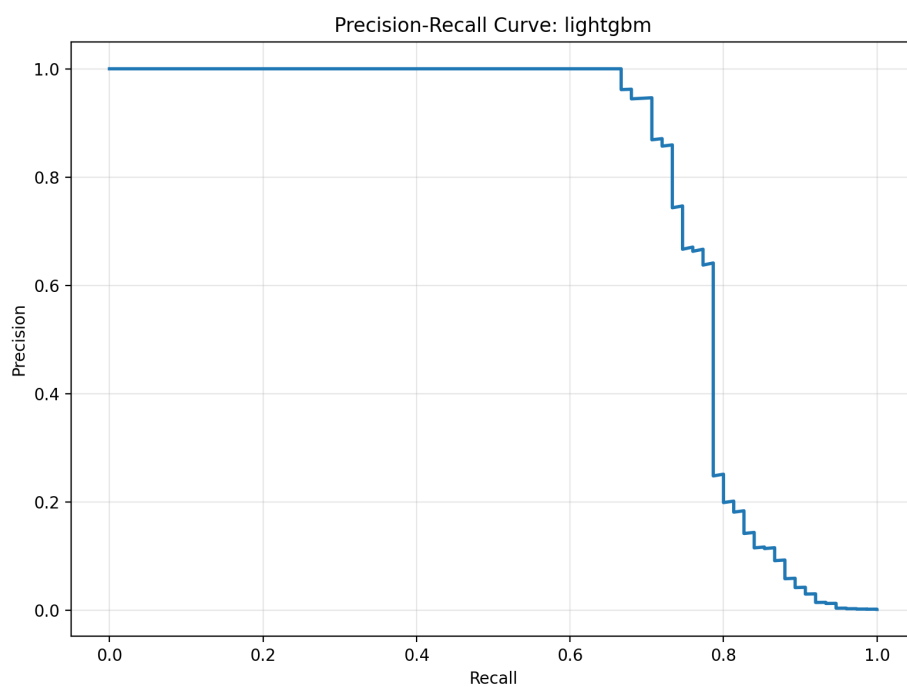


Figure 6.8: Precision-recall curve for the LightGBM run 20260518T034536Z-d5840edb

6.6 Analysis of Results

At the fixed decision threshold $\tau = 0.5$, the four model families show distinct operating behaviors on the imbalanced temporal holdout set. Logistic regression attains the highest recall (0.8800) but very low precision (0.0676), resulting in a low F1-score (0.1255). Random forest provides the strongest overall trade-off, with precision 0.9825, recall 0.7467, F1-score 0.8485, and average precision 0.8218.

XGBoost also performs strongly, with precision 0.8507 and recall 0.7600, yielding an F1-score of 0.8028 and average precision of 0.7830. LightGBM shows a similar but slightly weaker profile at this threshold, with precision 0.7534, recall 0.7333, F1-score 0.7432, and average precision 0.7811.

The main reason logistic regression appears weaker in F1 is the large false-positive volume at $\tau = 0.5$. It predicts 977 positives while the test set contains only 75 true fraud cases, so precision collapses. In contrast, the tree-based models capture nonlinear relationships and feature interactions, leading to a more favorable precision–recall balance under the same holdout protocol.

Therefore, logistic regression is not inherently unsuitable; rather, under the current threshold and configuration it generates too many false alarms. A fair follow-up is to perform systematic threshold tuning (and optionally probability calibration) before drawing final conclusions about model ranking.

Using this fixed-threshold view, the ranking by both F1-score and average precision is random forest first, followed by XGBoost, LightGBM, and logistic regression. In practical terms, random forest is the strongest default choice for this dataset when precision and overall balance are prioritized, while logistic regression can still be useful in recall-oriented screening scenarios if threshold tuning is applied.

Chapter 7

Discussion

This chapter interprets the obtained results, highlights project contributions, and discusses current limitations and learning outcomes.

7.1 Main Contributions of the Project

The main contribution of this project is the development of a pipeline-based system that makes the fraud detection workflow easier to execute and manage. Instead of training and evaluating models through scattered scripts or notebooks, the project organizes the process into a clear sequence of stages, including data preparation, model training, evaluation, hyperparameter sweep, and result comparison. In the current report version, this workflow is demonstrated across four model families (including gradient boosting methods), which broadens the empirical comparison beyond classical baselines. This design improves usability, reproducibility, and experiment management, allowing users to run and review machine learning experiments more efficiently.

7.2 Limitations

Despite its strengths, the project has several limitations. Four model families were compared, but systematic threshold optimization, feature engineering beyond scaling, and model explainability were not fully studied. Gradient boosting hyperparameters were not exhaustively tuned beyond the default sweep grids. In addition, the solution remains an offline batch workflow and is not designed as a production fraud monitoring service.

7.3 Learning Outcomes

Through the completion of this project, several technical skills were strengthened, including preprocessing for tabular data, evaluation of imbalanced classifiers, experiment management, artifact organization, and the presentation of machine learning results in a structured academic format.

7.4 Personal Contribution Summary

The primary contributions made in this project include the design of the preprocessing workflow, the implementation of training and evaluation stages, the introduction of run tracking and comparison artifacts, and the development of a dashboard for reviewing saved experiments. These contributions extend beyond simple model usage and reflect basic machine learning system design and engineering practice.

Chapter 8

Conclusion and Future Work

This chapter concludes the report by summarizing the main findings and outlining practical directions for improving the system in future work.

8.1 Conclusion

This report has presented the development of a reproducible credit card fraud detection pipeline for Internship 1. The implemented workflow covers data preparation, temporal splitting, feature scaling, oversampling, model training, evaluation, hyperparameter sweep, run comparison, and dashboard-based inspection. Across the four evaluated model families on the temporal holdout set, the random forest configuration achieved the strongest overall performance (highest F1-score and average precision), while logistic regression offered higher recall at the cost of a large number of false positives.

Taken as a whole, the project demonstrates an appropriate level of technical depth for Internship 1 by combining problem formulation, implementation, reproducibility, experiment management, and performance analysis within a single coherent workflow.

8.2 Future Work

Several directions may be considered for future improvement:

- performing systematic threshold tuning based on application-specific cost tradeoffs;
- incorporating explainability techniques (e.g., SHAP) for tree-based models;
- monitoring concept drift and evaluating rolling-window retraining strategies for time-evolving transaction patterns;
- extending the system into a service-oriented architecture or API;
- adding broader automated tests for pipeline reliability.

References

- [1] TensorFlow. Credit card fraud detection dataset mirror. <https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv>
- [2] scikit-learn developers. scikit-learn documentation. <https://scikit-learn.org/stable/>
- [3] imbalanced-learn developers. imbalanced-learn documentation. <https://imbalanced-learn.org/stable/>
- [4] Streamlit. Streamlit documentation. <https://docs.streamlit.io/>
- [5] scikit-learn developers. LogisticRegression API reference. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [6] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. <https://doi.org/10.1023/A:1010933404324>
- [7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. <https://doi.org/10.1613/jair.953>
- [8] Jesse Davis and Mark Goadrich. The relationship between Precision-Recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240, 2006. <https://doi.org/10.1145/1143844.1143874>
- [9] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012. <https://doi.org/10.1145/2382577.2382579>
- [10] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Learned lessons in credit card fraud detection from a practitioner perspective. *Expert*

- Systems with Applications*, 41(10):4915–4928, 2014. <https://doi.org/10.1016/j.eswa.2014.02.026>
- [11] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. <https://doi.org/10.1145/2939672.2939785>
- [12] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, 2017.
- [13] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009. <https://hastie.su.domains/ElemStatLearn/>

Appendix A

Example Commands

This appendix chapter provides representative command-line examples for reproducing the main training and evaluation workflows.

The following commands illustrate a typical execution flow:

```
python -m src.cli train --model xgboost
python -m src.cli evaluate --model xgboost
python -m src.cli train --model lightgbm
python -m src.cli evaluate --model lightgbm
python -m src.cli train --model all
python -m src.cli evaluate --model all
```